

CineExplorer

RACZKOWSKI Dawid (s200735) BUI Hoang Linh (s2503303)
DINH Duy Vu (s2401627)

Knowledge representation and reasoning, DEBRUYNE Christophe, University of Liège

May 6, 2026

Abstract

This report describes CineExplorer, a knowledge-graph application built over the official Internet Movie Database (**IMDb**) non-commercial dataset and deployed for SPARQL Protocol and RDF Query Language (**SPARQL**) querying and Linked Data browsing. The system covers the full Milestone 1–4 pipeline: a relational schema reduced from a 14-relation Entity-Relationship Diagram (**ERD**); a 1,611,676-triple knowledge graph generated by an RDB to RDF Mapping Language (**R2RML**) mapping over the migrated database; an Web Ontology Language (**OWL**) 2 Description Logic (**DL**) ontology centred on **CreativeWork** and a reified **Participation** class to capture per-credit context; and an Apache Fuseki deployment paired with the Brwsr Linked Data browser. Two non-trivial demonstrators close the loop. A Bacon-number analysis using **SPARQL** property paths reaches 92% of all 38,067 persons within three collaboration hops of Kevin Bacon, motivating the project’s earlier migration from a 174-title curated sample. A Shapes Constraint Language (**SHACL**) structural validation contrasts closed-world constraint checking with the open-world semantics of the **OWL** ontology, and surfaces a real 25-title mapping gap that the open-world view would silently allow. Together the artefacts demonstrate end-to-end use of database design, ontology engineering, **R2RML** mapping, **SPARQL** federation, and constraint-based Knowledge Graph (**KG**) validation.

Contents

1	Introduction	3
2	Dataset Description	3
2.1	The Official IMDb Non-Commercial Dataset	3
2.2	Meaningful Examples	4
2.3	Licensing	5
3	Towards an Information System	5
3.1	Entity-relationship Diagram	5
3.2	Reduction to Relational Models	7
3.3	Database Implementation	8
3.4	Meaningful Examples	9
4	Ontology Engineering	9
4.1	Scope	9
4.2	Concept Organization	9
4.2.1	Classes	9
4.2.2	Object Properties	11
4.2.3	Data Properties	11
4.3	OWL Profile	11
4.4	Namespace and IRIs	12
4.5	Encountered Problems	12

5	Annotating the Information System	13
5.1	Approach	13
5.2	IRI Strategy	13
5.3	Mapping Structure	14
6	Deployment	15
6.1	Architecture	15
6.2	Triplestore Setup	16
6.3	Loading the Knowledge Graph	16
6.4	SPARQL Endpoint	16
6.5	Linked Data Browser	17
6.6	Named Graph Strategy	17
7	SPARQL Queries	17
7.1	Query Overview	17
7.2	Q1 — Films with Genres and Runtime	17
7.3	Q2 — Persons per Participation Role	18
7.4	Q3 — Persons Who Worked Together Most	19
7.5	Q4 — Actors Spanning Three or More Genres	19
7.6	Q5 — Titles with No Director Credited	20
7.7	Q6 — Collaboration Distance via Property Paths	20
7.8	Q7 — Wikidata Federation	21
7.9	Q8 — Seasons and Episodes per Series	22
7.10	Q9 — Directors Who Never Acted	22
7.11	Q10 — Titles Sharing Genre and Language	23
8	Non-trivial Demonstrator	23
8.1	Bacon Number — Collaboration Graph Analysis	23
8.1.1	Concept	23
8.1.2	Source Person	24
8.1.3	Bacon Number Results	24
8.1.4	Path Reconstruction	24
8.2	SHACL Structural Validation	24
8.2.1	SHACL vs OWL Cardinality	24
8.2.2	Shapes Defined	25
8.2.3	Validation Results	25
8.2.4	Discussion	26
8.3	Summary	26
9	Conclusion	26
10	Member Contributions	27
A	Appendix: The use of AI	28
B	Additional Information on the Relational Model	28
B.1	Modelling Assumptions and Cardinalities	28
B.2	Notes on the Reduction	29
B.3	Attribute Domains	30
B.4	Description of Non-visible Integrity Constraints	31
C	Appendix: The Earlier Curated Sample	31

1 Introduction

CineExplorer is an exploration platform that integrates the **IMDb** non-commercial dataset [6] with a knowledge-graph architecture. The aim is to move beyond the row-and-column queries that **IMDb** itself supports, towards graph-shaped questions that exploit the connections between persons, titles, and genres. The application leans on the standard Semantic Web stack: an **OWL 2 QL** ontology defines the conceptual model, an **R2RML** mapping converts the relational source into Resource Description Framework (**RDF**), an Apache Fuseki triplestore serves **SPARQL** 1.1 queries, and an Apache Brwsr instance dereferences resource Internationalised Resource Identifiers (**IRIs**) as Linked Data [11]. Wikidata [12] is integrated through federated **SPARQL SERVICE** calls to enrich local persons with external attributes.

The report covers four milestones. Milestone 1 establishes the relational source: a 14-relation schema reduced from a conceptual **ERD**, populated from a numVotes-ranked top- N slice of the official **IMDb** dataset (Sections 2–3). Milestone 2 designs the ontology — centred on **CreativeWork** as the abstraction of all titles, and a reified **Participation** class that lets per-credit context (role, job, character names) hang off the relation rather than the endpoints (Section 4). Milestone 3 connects the two layers with an **R2RML** mapping that produces a 1,611,676-triple knowledge graph (Section 5). Milestone 4 deploys the result, evaluates ten **SPARQL** queries that exercise **SPARQL** 1.1’s main constructs (subqueries, federated **SERVICE**, property paths, aggregates, negation as failure), and presents two non-trivial demonstrators built on top: a Bacon-number collaboration analysis and a **SHACL** structural validation (Sections 6–8).

Three application directions guided the design throughout. First, the system supports collaboration analysis: graph-shaped questions of the form “how are these two persons connected through shared work,” formalised by the Bacon-number game and answered with **SPARQL** property paths. Second, it supports enrichment via federation: local persons are joined with Wikidata through their **IMDb** identifiers, without materialising any `owl:sameAs` bridge. Third, it supports Linked Data exploration: every resource in the **KG** is dereferenceable through Brwsr, so navigation can proceed without writing **SPARQL**.

2 Dataset Description

Earlier project milestones were prototyped on a third-party curated Comma-Separated Values (**CSV**) sample of **IMDb** [9, 10]. That sample was sufficient for the relational design and the initial ontology and mapping work, but a structural property of the slice prevented the Milestone 4 Bacon-number demonstrator from being exercised: no person in the curated sample appeared in more than one title, so the actor collaboration graph degenerated into disjoint single-title cliques and admitted no multi-hop expansion. To restore the connectivity the use case requires, the project migrated to **IMDb**’s official non-commercial release and rebuilt the database, ontology mapping, and knowledge graph against it. The earlier sample is preserved for reference in Appendix C; the rest of this section describes the official source.

2.1 The Official IMDb Non-Commercial Dataset

The upstream source is the **IMDb** non-commercial dataset [6], published by **IMDb** for personal and non-commercial use under their data-use terms [5]. It is distributed as seven gzip-compressed Tab-Separated Values (**TSV**) files refreshed daily, covering on the order of 12 million titles and 14 million persons. Two universal identifiers run through the entire release and become load-bearing in our design: `tconst` (title constant, e.g. `tt0073195`) and `nconst` (name constant, e.g. `nm0000102`). They appear as foreign keys throughout the relational design (Section 3) and as the **IRI** templates of the knowledge graph (Section 5).

The seven files and their roles in the CineExplorer relational schema are summarised in Table 2.1.

File	Key columns	Role in our schema
<code>title.basics</code>	<code>tconst</code> , <code>primaryTitle</code> , <code>titleType</code> , <code>startYear</code> , <code>runtimeMinutes</code> , <code>genres</code>	Identifies titles; primary feeder for the <code>title</code> relation.
<code>title.akas</code>	<code>titleId</code> , <code>ordering</code> , <code>region</code> , <code>language</code> , <code>types</code> , <code>attributes</code>	Localised titles for <code>title_aka</code> .
<code>title.crew</code>	<code>tconst</code> , <code>directors</code> , <code>writers</code>	Pre-aggregated director and writer lists; expanded into rows of <code>talent_title</code> .
<code>title.episode</code>	<code>tconst</code> , <code>parentTconst</code> , <code>seasonNumber</code> , <code>episodeNumber</code>	Series-to-episode parentage for <code>title_episode</code> .
<code>title.principals</code>	<code>tconst</code> , <code>ordering</code> , <code>nconst</code> , <code>category</code> , <code>job</code> , <code>characters</code>	One row per credited principal — source of <code>title_principal</code> and the playable-role data.
<code>title.ratings</code>	<code>tconst</code> , <code>averageRating</code> , <code>numVotes</code>	Popularity signal; also drives the slicing criterion in Section 2.2.
<code>name.basics</code>	<code>nconst</code> , <code>primaryName</code> , <code>birthYear</code> , <code>deathYear</code> , <code>primaryProfession</code> , <code>knownForTitles</code>	The person catalogue, source of <code>talent</code> .

Table 2.1: The seven **TSV** files of the **IMDb** non-commercial release and their role in the CineExplorer relational schema. `tconst` and `nconst` are the canonical keys for titles and persons throughout.

The release ships pre-aggregated comma-separated lists in three places (`genres`, `directors`, `writers`, `knownForTitles`); our relational design normalises each of these into a junction table, restoring 1NF and recovering the join paths that the ontology mapping depends on. The full normalisation is described in Section 3.

2.2 Meaningful Examples

The full **IMDb** release is too large to deploy on a laptop, and its long tail consists overwhelmingly of obscure works with very few ratings. We therefore work on a controlled slice that preserves the dense head of the distribution and is sufficient to exercise every component of the platform end-to-end. The slice serves as the running set of meaningful examples throughout the rest of this report.

Slice criterion. The slice is defined by a single rule: rank all rated titles by `numVotes` in descending order, take the top N . Default $N = 5,000$. Two properties matter for the use case:

1. **Popularity-driven cut-off.** **IMDb**’s metadata is densest for popular productions — they accumulate more credited principals, more genre tags, and more alternative-title rows. The slice concentrates the credit mass that the actor collaboration graph depends on at the head of the distribution.
2. **Determinism.** A re-download of `title.ratings` reproduces the same top- N identifiers exactly, so the slice is fully reproducible from the same dataset snapshot. The threshold is a function of the data, not a manual choice.

The alternatives we considered — a uniform random sample, a date-range cut, or a hand-picked seed expanded by graph traversal — each gave up at least one of these properties. Random sampling diluted the collaboration density; a date cut over-represented one era; a seeded crawl made the slice impossible to reproduce. The `numVotes`-ranked top- N rule is the simplest cut that preserves both. A global cut-off naturally yields a film-heavy mix (films attract more votes than individual TV episodes), but the resulting type distribution is itself informative — the slice reflects the popularity skew that motivates the use case.

Slice statistics. At $N = 5,000$, the resulting slice contains:

Source file	Rows after slice	Notes
<code>title.basics</code>	5,000	joined on top- N tconsts
<code>title.ratings</code>	5,000	ranking source for the slice
<code>title.principals</code>	105,964	all credit rows for retained titles
<code>title.crew</code>	5,000	retained for FK closure
<code>title.episode</code>	139	only those with parent in slice
<code>title.akas</code>	256,858	only for retained titles
<code>name.basics</code>	38,067	only persons referenced by retained titles

Table 2.2: Slice statistics for $N = 5,000$. Joined rows from `title.basics` and `title.ratings` populate the `title` relation; `title.principals` feeds the `principal` weak entity.

Within the 5,000 retained titles, the type breakdown is: `movie` 4,214, `tvSeries` 542, `tvEpisode` 139, `tvMiniSeries` 80, `video` 9, `short` 5, `tvMovie` 5, `tvSpecial` 3, `videoGame` 3. Films dominate, as expected from a popularity-ranked global cut-off.

Connectivity diagnostic. The slice yields 13,521 `bridge persons` (persons credited on two or more titles), well above the threshold of 100 required for a meaningful Bacon expansion. Each person averages 2.78 credits across the slice, and the resulting collaboration graph is a single large connected component covering 92% of all persons within three hops of Kevin Bacon (Section 8).

Pipeline. The slicing pipeline lives under `database/etl/` and consists of three Bash stages: `download.sh` (idempotent fetch and decompression of the seven **TSVs**), `filter_top_n.sh` (global `numVotes` ranking, top- N tconst selection, joins through the seven **TSVs**, sentinel-guarded so re-running does not recompute), and `connectivity_diagnostic.sh` (sanity checks on collaboration density before loading). The MySQL Extract, Transform, Load (**ETL**) itself is a single all-in-one script `database/imdb-schema.sql`, organised as six in-session phases from canonical-table Data Definition Language (**DDL**) through staging-table loaders to post-load FKs and indexes (Section 3.3).

2.3 Licensing

The **IMDb** data remains subject to **IMDb**’s non-commercial usage conditions [5]. The slicing scripts and the relational schema we author around them are released under the project’s MIT license.

3 Towards an Information System

This section presents the conceptual **ERD**, the modelling assumptions and cardinalities, the reduction to a relational schema, and the database that materialises it. The schema described here is the migrated CineExplorer schema built directly from the official **IMDb** non-commercial dataset (Section 2). Earlier project milestones used a curated **CSV** sample with a coarser schema; that schema is preserved in the repository’s archive but is not reused below.

3.1 Entity-relationship Diagram

Figure 3.1 illustrates the **ERD**. Two strong top-level entity sets (`Title` and `Person`) anchor the model, surrounded by six lookup entity sets (`Title_Type`, `Role`, `Profession`, `Language`, `Region`, `AKA_Type`) for the bounded label sets used in the source **TSVs**. Two weak entity sets (`Title_AKA` and `Principal`) capture relations that carry their own attributes and require an identifying parent. Three IS-A subtypes of `Title` (`Film`, `Series`, `Episode`) match the three disjoint subclasses of `ce:CreativeWork` declared in the ontology. A small number of multi-valued attributes (`Genre` on `Title`, `character_name` on `Principal`) are reduced to attribute relations rather than promoted to entity sets, because the value sets are small, fully bounded, and the value string is its own complete label. The full reduction yields 16 relations (Appendix B).

A note on naming. Principal is named after **IMDb**’s source vocabulary — the file is `title.principals.tsv` and each row is a ”principal credit.” At the ontology layer (Section 4) the same concept is named `ce:Participation` (the abstract reified-relationship name). The **R2RML** mapping bridges the two: it reads the Structured Query Language (**SQL**) `principal` table and emits `ce:Participation` instances. We adopt the two-layer naming because the relational schema benefits from source fidelity (anyone reading **IMDb**’s data dictionary maps directly onto our tables), while the ontology benefits from naming a reified n -ary relationship after the relationship verb rather than after one source’s term.

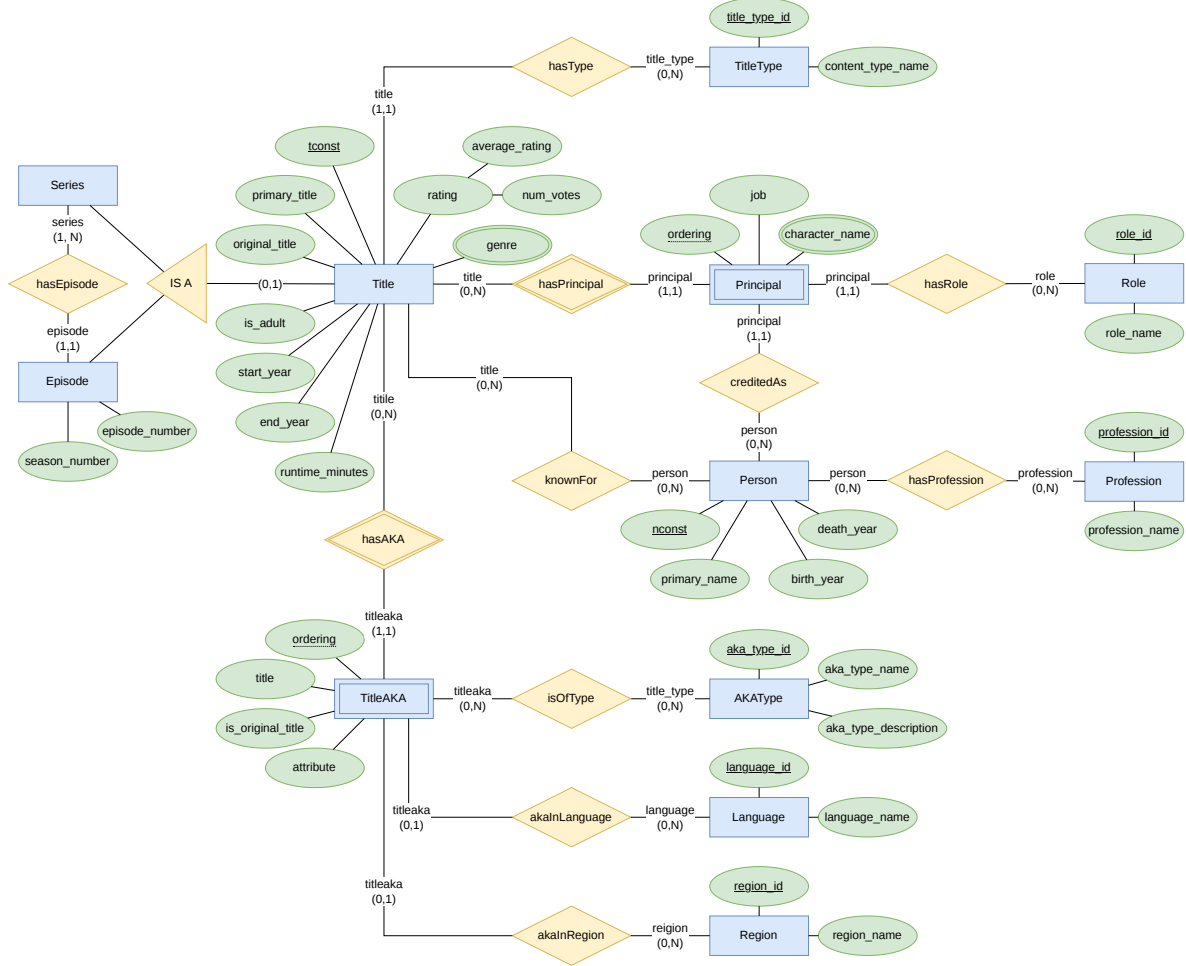


Figure 3.1: **ERD** for the migrated CineExplorer schema. **Title** and **Person** are strong entity sets; **Film**, **Series**, **Episode** are partial-disjoint IS-A subtypes of **Title**; **Title.AKA** and **Principal** are weak entity sets; **Title.Type**, **Role**, **Profession**, **Language**, **Region**, **AKA.Type** are lookup entity sets.

The central entity set **Title** represents an audiovisual work. It carries identifier (`tconst`, **IMDb**’s natural key), descriptive attributes (`primary_title`, `original_title`, `is_adult`, `start_year`, `end_year`, `runtime_minutes`), and a popularity composite (`average_rating`, `num_votes`). Each **Title** is classified by exactly one **Title.Type** (e.g., `movie`, `tvSeries`, `tvEpisode`); the `title_type_id` FK serves both as a classification and as the discriminator for the IS-A specialisation. **Series** is the IS-A subtype induced by `title_type_id ∈ {tvSeries, tvMiniSeries, tvPilot}`; **Episode** by `title_type_id = tvEpisode`; **Film** by `title_type_id = movie`. The specialisation is partial: titles whose type does not fall in this triad (for example `short`, `video`, `tvSpecial`) remain plain **Title** rows with their classification preserved but no IS-A subtype membership. Only **Episode** stores extra attributes (`parent.tconst`, `season_number`, `episode_number`); **Film** and **Series** have no per-subtype attributes and so reduce to predicate views over **Title**.

Person represents people credited on titles, identified by **IMDb**'s **nconst**. Two distinct relationships connect persons to titles. The first is per-credit: an n-ary association between **Person**, **Title**, and **Role** is reified into the weak entity set **Principal**, identified by (**tconst**, **ordering**) from the source **title.principals TSV**. **Principal** carries the per-credit attributes **role_id** (FK to **Role**), **job** (free-text job description), and a multi-valued **character_name** reduced to the attribute relation **Principal_Character**. The second is career-level: a person's **IMDb** **primaryProfession** is a multi-valued attribute (capped at three values per person) modelled as the M:N junction **Person_Profession** between **Person** and the **Profession** lookup. A person credited as **director** on one title and **actor** on another therefore has two **Principal** rows but may have one or both professions in **Person_Profession** depending on what **IMDb** records as their career-level identification. We discuss this two-axis modelling and its consequences for the ontology below (Section 4.5).

Genre is modelled as a multi-valued attribute on **Title** rather than as a lookup entity. The genre vocabulary is small (26 distinct values in the slice), bounded, and the value string is its own label (**Drama**, **Comedy**), so reducing it to the attribute relation **Title_Genre(tconst, genre_name)** preserves all expressiveness without an extra lookup table. **Title_AKA** is a weak entity set with composite key (**tconst**, **ordering**) that records alternative regional or linguistic titles; it links optionally to **Language** and **Region** lookups, and to a sixth lookup **AKA_Type** (whose 8-value vocabulary – **imdbDisplay**, **original**, **alternative**, **tv**, **dvd**, **video**, **festival**, **working** – carries **IMDb**-shorthand codes that benefit from a description column, so a flat multi-valued attribute would lose the explanation). The link is M:N because **IMDb**'s **title.akas.types** column may carry several codes per **AKA** row; the **Title_AKA_Type** junction reduces the relation. The series-episode hierarchy is encoded in the **title_episode** table, which adds the **parent_tconst** self-FK from **Episode** back to **Title** (the parent series) together with **season_number** and **episode_number** for ordered navigation.

3.2 Reduction to Relational Models

The **ERD** reduces by the standard rules to the 16 relations listed below, grouped by reduction category:

% Strong entity sets (8 relations)

```
Title(tconst, primary_title, original_title, is_adult,
      start_year, end_year, runtime_minutes,
      average_rating, num_votes, title_type_id)
```

```
Person(nconst, primary_name, birth_year, death_year)
```

```
Title_Type(title_type_id, title_type_name)
```

```
Role(role_id, role_name)
```

```
Profession(profession_id, profession_name)
```

```
Language(language_id, language_name)
```

```
Region(region_id, region_name)
```

```
AKA_Type(aka_type_id, aka_type_name, aka_type_description)
```

% IS-A subtype with extra attributes (1 relation)

```
Title_Episode(tconst, parent_tconst, season_number, episode_number)
```

% Weak entity sets (2 relations)

```
Title_AKA(tconst, ordering, title, region_id, language_id,
          is_original_title, attributes)
```

```
Principal(tconst, ordering, nconst, role_id, job)
```

% Multi-valued attributes and junctions (5 relations)

```
Title_Genre(tconst, genre_name)
```

```
Principal_Character(tconst, ordering, character_name)
```

```
Person_Profession(nconst, profession_id)
```

```
Person_Known_For(nconst, tconst)
```

```
Title_AKA_Type(tconst, ordering, aka_type_id)
```


Primary keys are the leading attributes of each signature. The full listing with attribute domains, foreign keys, and integrity constraints is in Appendix B.

3.3 Database Implementation

The database is built in four stages: an **SQL** translation of the relational model, a filtering pipeline that prepares the data, a containerised MySQL deployment, and a loading step that materialises the filtered data into the canonical schema.

1. SQL schema. The 16 relations of Section 3.2 are translated into MySQL in the natural way: each relation becomes a table, primary keys become **PRIMARY KEY** declarations, weak entity sets and M:N junctions receive composite primary keys, and the foreign keys recovered by reduction become **FOREIGN KEY** constraints on the appropriate columns. **Title.tconst** and **Person.nconst** retain **IMDb**’s natural string keys (the source data already provides canonical identifiers); the remaining lookup tables use **INT AUTO_INCREMENT** surrogate keys, except for **Title.Type**, **Language**, and **Region**, which keep **IMDb**/ISO code strings as primary keys. The character set is **utf8mb4** throughout so that non-Latin titles (Japanese, Cyrillic, etc.) round-trip correctly.

2. Filtering. The raw **IMDb** non-commercial release ships seven gzipped **TSV** files refreshed daily. A small Bash pipeline downloads the latest release, applies the numVotes-ranked top-*N* rule of Section 2.2, and joins the resulting tconst set back through the remaining **TSVs** to produce a filtered, FK-closed snapshot on disk. The output is seven smaller **TSVs** that share **IMDb**’s column shape but contain only the rows reachable from the top-*N* titles. The pipeline is sentinel-guarded, so re-running against an already-prepared snapshot is a no-op.

3. Containerised deployment. The database engine and an admin UI are deployed together via Docker Compose: a pinned MySQL container as the data engine, alongside phpMyAdmin for browsing and ad-hoc inspection. Containerisation pins the engine version across team machines, isolates the project database from any host MySQL installation, and lets the filtered-**TSV** directory be bind-mounted into the container so the engine reads the source files in place rather than through a copy. A single **docker compose up** brings the engine and the admin UI online with FK-strict mode and **utf8mb4** as defaults.

4. Loading. Loading proceeds via a staging-tables pattern: the seven filtered **TSVs** are first bulk-loaded into disposable staging tables that mirror **IMDb**’s column shape exactly, then transformed into the canonical tables of stage 1 by **INSERT ... SELECT** statements that apply 1NF reduction and three normalisations:

- **Multi-valued attribute explosion.** Comma-separated and JSON-encoded fields in the source (**title.basics.genres**, **title.principals.characters**, **name.basics.primaryProfession**, **title.akas.types**) are unrolled into 1NF junction rows so that each multi-valued attribute lands in its own dedicated relation. The **Adult** value of the genre vocabulary is dropped at this step because the boolean **is_adult** column on **title** already carries the same information.
- **actor/actress collapse.** **IMDb**’s gender-split per-credit category is unified to **actor** at INSERT time; the gender axis is irrelevant to the use case and would otherwise complicate **SPARQL** aggregations. The decision is documented in the ontology comment on **ce:Actor**.
- **Foreign-key closure on person_known_for.** **IMDb** lists up to four notable titles per person (**name.basics.knownForTitles**); references whose target falls outside the slice are dropped at INSERT time, so every FK from **person_known_for.tconst** to **title.tconst** resolves to a row that is actually present.

The staging tables are dropped after the canonical tables are populated, so the final database holds only the canonical schema. Foreign keys and secondary indexes are added as the last step — after the canonical tables are filled — which lets the bulk load complete in a single pass without per-row constraint validation. The deterministic slice (Section 2.2) combined with the deterministic

transformation chain means re-running the four stages against the same dataset snapshot reproduces the database byte-for-byte.

3.4 Meaningful Examples

The migrated database, populated from the $N = 5,000$ global numVotes-ranked slice (Section 2), contains 5,000 titles, 38,067 persons, 105,964 principal credit rows, 256,858 alternative-title (`title_aka`) rows, and 139 individual episodes — a roughly $200\times$ scale-up over the previous 174-title curated CSV sample on the strong entity sets, with a much wider geographical/linguistic spread on alternative titles. Spot-check examples include the cast and crew of *Les Misérables* (2012), the entire Marvel Cinematic Universe in the slice (where multiple films share both genres and credited persons), and the cult-series subset of *Game of Thrones* and *Breaking Bad* for which several individual episodes cleared the global numVotes threshold and were therefore retained.

4 Ontology Engineering

The CineExplorer ontology provides a conceptual model for representing creative works and the persons involved in them. It is designed to support the annotation of the relational database (Section 3) and the construction of the RDF knowledge graph deployed in Section 6.

4.1 Scope

The ontology supports three main application directions. It enables actor connectivity analysis through path queries between persons connected via shared creative works. This functionality is enabled by the structure of the RDF graph, where persons are linked to works through participation relations.

It also provides a conceptual structure for aligning local entities with external resources such as Wikidata [12]. In addition, it supports graph-based exploration of related entities through navigation across persons, creative works, genres, and participation relations.

The ontology therefore covers the concepts required for these use cases: **CreativeWork** instances (**Film**, **Series**, and **Episode**), **Person**, **Genre**, descriptive attributes such as language and region, and contextual participation relations. This structure also supports interface-level exploration driven by the knowledge graph.

4.2 Concept Organization

The ontology is organized into classes, object properties, and data properties. This separation maintains a clear distinction between entities, relations, and literal attributes.

The ontology structure is visualized in Figure 4.2 using WebVOWL [7].

4.2.1 Classes

The central class is **CreativeWork**, which represents a work independently of its textual representation in the dataset. This abstraction separates the notion of a work from its titles.

CreativeWork is specialized into **Film**, **Series**, and **Episode**. These subclasses are declared mutually disjoint. The class **Person** represents individuals involved in productions and is specialized into **Actor**, **Director**, **Writer**, **Editor**, and **Composer**. These subclasses are not declared disjoint, as multiple roles may be associated with the same individual.

The ontology is informed by the structure of the underlying relational schema. Class and property names correspond to elements present in the dataset, which simplifies the mapping and ensures interpretability. At the same time, the ontology introduces abstraction layers that are not present in the database. In particular, the use of **CreativeWork** instead of **Title** and the introduction of **Participation** reflect conceptual modeling decisions rather than direct schema replication.

The class **Participation** reifies the relation between a **Person** and a **CreativeWork**. This design allows contextual attributes such as role and character name to be associated with the relation itself.

Participation is declared disjoint from **CreativeWork**, **Person**, and **Genre**. Its is connected to exactly one **Person** through `playedBy` and exactly one **CreativeWork** through `participatesIn`.

Episode is associated with a **Series** through `partOfSeries`.

4.2.2 Object Properties

Object properties define relations between entities. These include relations between works and genres, relations between episodes and series, and relations between persons and works through **Participation**.

The properties **workedFor** and **employed** define a general relation between **Person** and **CreativeWork**. More specific properties such as **actedIn**, **directed**, **wrote**, **edited**, and **composedFor** are defined as sub-properties. Each property is associated with an inverse to support bidirectional navigation.

The property **workedWith** represents a relation between persons derived from shared participation in the same creative work. It is primarily a traversal-oriented relation and can be derived from participation links, but may also be materialized to simplify querying.

The property **sameAsExternal** links local resources to external identifiers. No cardinality restriction is imposed on this property.

4.2.3 Data Properties

Data properties associate entities with literal values. **Person** is described by attributes such as name and life dates, plus a multi-valued **ce:hasProfession** (**xsd:string**, up to three values per person) that captures the **IMDb**-level career-profession axis introduced by the migration. **CreativeWork** is described by attributes such as title, release year, runtime, content flags, and the popularity composite **ce:averageRating** / **ce:numVotes** that drives the slice criterion (Section 2). **Participation** includes attributes describing roles and character information. **Episode** includes season and episode numbers.

Language and region are represented as literal attributes of **CreativeWork**. This choice keeps the ontology simple and consistent with their descriptive role in the dataset.

4.3 OWL Profile

The ontology targets the **OWL 2 QL** profile [8]. Of the four **OWL 2** sub-profiles (**DL**, **EL**, **QL**, **RL**), **QL** is the one designed for query rewriting over relational databases, with reasoning in AC^0 with respect to the data. This is a natural fit for an **R2RML**-backed deployment where the relational source is the authoritative store and the knowledge graph is large (1.6 M triples at $N = 5,000$).

The constructs actually used by the ontology all fall within **QL**:

- Subclass and sub-property axioms between named classes and properties (e.g., **Episode** $\sqsubseteq$ **CreativeWork**, **actedIn** $\sqsubseteq$ **workedFor**).
- Inverse object properties (**owl:inverseOf**) for bidirectional navigation.
- Symmetric property declarations (**owl:SymmetricProperty**) for **ce:workedWith**.
- Disjointness axioms over named classes (**owl:disjointWith**, **owl:AllDisjointClasses**) for the pairwise disjointness of **Film**, **Series**, and **Episode**, and for the disjointness of **Person**, **CreativeWork**, **Genre**, and **Participation**.
- Domain and range axioms on object and datatype properties.

The constructs we deliberately do not use are precisely those that would push the ontology out of **QL**: cardinality restrictions (qualified or unqualified), functional and inverse-functional property declarations, transitive properties, and property chains. Where structural cardinality is genuinely required — for example the “exactly one Series per Episode” invariant or the “exactly one Person and one CreativeWork per Participation” constraint — it is encoded as a **SHACL** shape rather than as an **OWL** axiom. The reasoning follows the open-world / closed-world distinction: **OWL** operates under the open-world assumption, which makes it unsuitable for checking whether a knowledge graph contains all required triples; **SHACL** validates the graph against a shapes graph under a closed-world assumption, so an absent required triple is reported as a violation.

Profile membership was confirmed using Protégé’s **OWL** Profile Checker. We additionally verified consistency using the HermiT reasoner [2]: classification computes all inferred subsumption relations between named classes and checks whether any class is unsatisfiable. No unsatisfiable classes were found, confirming that the ontology is consistent.

4.4 Namespace and IRIs

The ontology uses a hash Uniform Resource Identifier (**URI**) pattern [11] with a single namespace: `http://cineexplorer.local/ontology#`. Under this pattern, each ontology term is identified by a **URI** fragment (e.g., `http://cineexplorer.local/ontology#Film`), and dereferencing any term **IRI** resolves to the ontology document. This avoids the HTTP 303 redirect infrastructure required by slash **URI** patterns, making it appropriate for a locally deployed academic project where a publicly accessible server is not available.

Instance **IRIs** follow a separate path-based structure: `http://cineexplorer.local/data/{type}/{id}`, reusing the stable **IMDb** identifiers from the source dataset as local names. The **IRI** design is described further in Section 5.

Class and property names follow the naming conventions used throughout the W3C **OWL** vocabulary [3]: UpperCamelCase for class names (e.g., **CreativeWork**, **Participation**) and lowerCamelCase for property names (e.g., **workedFor**, **partOfSeries**). All terms carry `rdfs:label` and `rdfs:comment` annotations in English.

4.5 Encountered Problems

Tooling. The ontology was developed in Protégé 5.6.9 using the Hermit reasoner for consistency checking and classification. An early version of the ontology triggered an unsatisfiability due to a conflicting disjointness axiom on **Participation**; this was resolved by narrowing the disjointness to the intended classes only.

Property directionality. **R2RML** generates triples in the direction defined by the mapping template. A property such as `ce:actedIn (Person → CreativeWork)` requires a separate triple map for its inverse `ce:hasActor` if queries need to traverse from work to person. Explicit inverse properties were therefore introduced for each role-specific property to avoid requiring `OPTIONAL { ?w ^ce:actedIn ?p }` patterns in every query.

Participation vs. direct properties. An initial design used only direct role-specific properties between **Person** and **CreativeWork**. The **Participation** reification was introduced to allow contextual attributes — character name, job description, credit order — to be attached to the relation itself rather than to either endpoint. This decision increased the number of triple maps but improved the expressiveness of the model.

Cardinality, the open-world assumption, and the move to SHACL. Early drafts encoded structural cardinalities as **OWL** axioms: `owl:qualifiedCardinality "1"` restrictions on `ce:partOfSeries` (**Episode**), on `ce:participatesIn/ce:playedBy` (**Participation**), and `owl:maxCardinality "1"` on `ce:sameAsExternal` (**Person**, **CreativeWork**). The functional characteristic was also declared on `ce:partOfSeries`. These axioms placed the ontology in **OWL 2 DL**. They were removed in favour of **SHACL** shapes for two reasons. First, **OWL** cardinalities define class-membership semantics under the open-world assumption rather than data-completeness constraints: an **Episode** individual with no recorded `partOfSeries` triple does not violate `owl:qualifiedCardinality "1"` — the reasoner simply has no information about its series. What we actually want is a closed-world integrity check over the materialised graph, which is precisely what **SHACL** provides. Second, removing these axioms moves the ontology into **OWL 2 QL**, opening up query-rewriting deployments and giving AC0 reasoning complexity (Section 4.3). The slice-induced invariant that every **CreativeWork** must carry an `averageRating` and a `numVotes` value follows the same pattern and is also encoded as a **SHACL** shape.

Two axes for “role” vs. “profession”. The migration introduced two related but distinct axes of person classification: per-credit role (the `role` on each **Participation**, e.g., this row is an actor credit on this title) and career-level profession (the **IMDb** `primaryProfession`, e.g., this person is identified as a director-and-producer overall). The ontology mirrors this split. **Participation** carries `ce:participationRole` as a per-credit string. **Person** carries `ce:hasProfession` as the multi-valued career attribute. The **Person** subclasses (**Actor**, **Director**, **Writer**, **Editor**, **Composer**) are populated from either axis: an individual is asserted as `ce:Director` either because they have a director credit

on some **Participation**, or because their **IMDb** **primaryProfession** contains **director**. Keeping the two axes separate at the data-property level lets queries distinguish “someone with at least one director credit” from “someone **IMDb** identifies primarily as a director”, a distinction the previous schema could not express.

5 Annotating the Information System

This section describes how the relational database is mapped to **RDF** triples using **R2RML** [1], producing an instance of the CineExplorer knowledge graph conforming to the ontology described in the previous section.

5.1 Approach

R2RML (RDB to **RDF** Mapping Language) [1] is used as the mapping language. **R2RML** is a W3C recommendation that defines how relational tables and **SQL** queries can be mapped to **RDF** graphs. The mapping file `cineexplorer_mapping.ttl` is serialized in Turtle syntax and processed by the `r2rml.jar` tool used in the lab sessions. The generated **RDF** output is written to `output/cineexplorer_kg.ttl` in Turtle format.

The mapping is executed with the following command from the project root:

```
java -jar tools/r2rml/r2rml.jar mapping/mapping.properties
```

where `mapping.properties` provides the JDBC connection string, credentials, input mapping file path, and output file path.

The mapping strategy follows the conceptual structure of the ontology rather than directly mirroring the table structure. In particular, the mapping is organized around the main ontology entities (**CreativeWork**, **Person**, **Participation**, and **Genre**) and around a small number of additional triples maps used to enrich these entities with structural or descriptive information.

The mapping consumes the canonical schema of Section 3 as published, with no further view layer. Multi-valued attributes (genres on **Title**, character names on **Participation**, professions on **Person**) are already 1NF-flattened into dedicated relations during the **SQL ETL** (Section 3.3), which means the **R2RML** side does not need any string-splitting tricks: each multi-valued attribute is one logical table and one triples map. This is a deliberate division of responsibilities — the relational layer handles 1NF, the **R2RML** layer handles class-and-property assignment — and matches the textbook **ERD** reduction discussed in the lectures.

5.2 IRI Strategy

A consistent **IRI** strategy was defined to ensure that each entity has a stable identifier. **IRIs** reuse the **IMDb** identifiers already present in the source data: `tconst` values for titles and `nconst` values for persons. These identifiers are stable, globally recognised, and make it unnecessary to introduce any additional keys into the schema. All **IRIs** share the project’s local namespace, `http://cineexplorer.local/data/`.

The main **IRI** templates used are:

- **Creative works** (Film, Series, Episode):
`http://cineexplorer.local/data/title/{TCONST}`
where `TCONST` is the **IMDb** `tconst` (e.g., `tt0111161` for The Shawshank Redemption).
- **Persons**:
`http://cineexplorer.local/data/person/{NCONST}`
where `NCONST` is the **IMDb** `nconst` (e.g., `nm0000128` for Russell Crowe).
- **Genres**:
`http://cineexplorer.local/data/genre/{GENRE_NAME}`
where `GENRE_NAME` is the genre string itself (e.g., `Drama`). The genre vocabulary is small, bounded, and the name is its own identifier in **IMDb**’s data, so no surrogate key is needed.

- **Participation nodes:**

`http://cineexplorer.local/data/participation/{TCONST}/{ORDERING}`

where the composite key (TCONST, ORDERING) matches the primary key of the **SQL** principal table (the relational schema uses **IMDb**'s source vocabulary; the ontology layer uses `ce:Participation` for the same concept — see Section 3). ORDERING is the per-title credit position from `title.principals`, which uniquely identifies one credit row even when the same person holds more than one credit on the same title (the ORDERING differs across credits). NCONST is intentionally not embedded in the **IRI**: a participation **IRI** keeps its identity even if the credited person is corrected upstream, and the `playedBy` edge still recovers the link.

The migration tightened this scheme relative to earlier drafts. The previous Participation **IRI** template used the triple (TITLE_ID, TALENT_ID, ORD), but with the official **IMDb** data it became clear that ORD alone (which **IMDb** calls `ordering`) already disambiguates within a title, and folding NCONST into the **IRI** duplicates information already available through `ce:playedBy`. The two-key form `{TCONST}/{ORDERING}` is also exactly the canonical primary key of the **SQL** principal relation, which keeps the **IRI** strategy aligned with the relational schema.

5.3 Mapping Structure

The mapping is organized by entity type and by the role played by each triples map in the generation of the **RDF** graph.

Three triples maps generate the three subclasses of `CreativeWork`. The discriminator is the `TITLE_TYPE_ID` column on `title`: Film corresponds to `TITLE_TYPE_ID = 'movie'`, Series to `TITLE_TYPE_ID IN ('tvSeries', 'tvMiniSeries', 'tvPilot')`, and Episode to `TITLE_TYPE_ID = 'tvEpisode'` (with the `title_episode` join supplying `partOfSeries`, `seasonNumber`, and `episodeNumber`). Each map carries the popularity composite (`ce:averageRating` from `AVERAGE_RATING`, `ce:numVotes` from `NUM_VOTES`) and the descriptive data properties `ce:workTitle`, `ce:originalTitle`, `ce:releaseYear`, `ce:runtimeMinutes`, and `ce:isAdult`. The data properties are individually predicated, so a NULL in one column skips only that triple, not the entire subject row, as required by **R2RML** §10.3 [1].

Title types outside the `movie / tvSeries+tvMiniSeries+tvPilot / tvEpisode` triad (e.g., `short`, `video`, `tvMovie`, `tvSpecial`, `videoGame`) remain in the `title` relation but receive no `rdf:type` assertion: in the slice they account for 25 of 5,000 titles. They still appear in the **KG** as objects of `ce:knownFor` and `ce:participatesIn` (hence inferred to `ce:CreativeWork` under **RDF** Schema (**RDFS**)), which motivates either extending the mapping to cover them or filtering them out at **ETL** time.

All rows from the `person` table are mapped to `ce:Person` with their descriptive data properties (`ce:personName`, `ce:birthYear`, `ce:deathYear`). Role-specific subclass membership (`Actor`, `Director`, `Writer`, `Composer`, `Editor`) is populated from two sources: the per-credit principal table (joined with `role`) and the career-level `person_profession` table (joined with `profession`). Both feeds use the same person-IRI template, so the resulting subclass-typing triples merge on the same subject. This two-axis approach (Section 4.5) means a person can be typed as `ce:Director` either because they have a director credit on some principal row, or because their **IMDb** `primaryProfession` contains `director`, or both. The `ce:hasProfession` datatype property is also emitted from `person_profession`, so the career axis is queryable as a string attribute as well.

The Participation triples map selects from the **SQL** principal table (joined with `role` for the role-name string) and generates one `ce:Participation` instance per credit row. Each instance is linked to its person via `ce:playedBy` and to its work via `ce:participatesIn`. The role name is stored as `ce:participationRole` and the free-text job description as `ce:participationProperties`. Each instance carries an `rdfs:label` constructed from the role name and the **IMDb** identifiers of the person and title, providing a human-readable identifier for use in **SPARQL** results and Linked Data browsers.

Character names are drawn from the **SQL** `principal_character` table and mapped by a dedicated triples map targeting the same participation **IRI** template, adding `ce:characterName` triples without duplicating participation nodes.

Cross-entity links such as `ce:partOfSeries` and `ce:hasGenre` are expressed using matching `rr:template` patterns on both sides rather than `rr:RefObjectMap` with explicit join conditions. This is possible because all entities of the same type share a single **IRI** template: any map that knows a

`TITLE_ID` can construct the correct title **IRI** without needing a formal join. The advantage is simplicity — no join overhead and no dependency on the logical table structure. The disadvantage is tight coupling: if the **IRI** template for one entity type were to change, every map that references it via template would also need to be updated. An `rr:RefObjectMap` approach would localise that change to a single parent map.

Genres are stored relationally as a multi-valued attribute (`title_genre(TCONST, GENRE_NAME)`), without a separate `genre` entity table. Two triples maps over `title_genre` produce the **RDF** representation: one creates each `ce:Genre` node from `DISTINCT GENRE_NAME` (giving 26 instances in the slice), the other emits the `ce:hasGenre` edge from each `Title` to its genre. Both maps share the **IRI** template `.../data/genre/{GENRE_NAME}`, so the two layers naturally merge on the same subjects.

The `person_known_for` table, which records up to four notable works associated with a person (sourced from **IMDb**'s `name.basics.knownForTitles`), is mapped to `ce:knownFor` links from person **IRIs** to title **IRIs**. FK closure is enforced at **ETL** time (Section 3.3), so every emitted `ce:knownFor` object resolves to a title that is actually present in the **KG**.

Besides the participation-centered representation, the mapping also materializes several direct properties between persons and creative works, such as `workedFor` and `employed`, together with more specific sub-properties such as `actedIn`, `directed`, `wrote`, `edited`, and `composedFor`. These properties are redundant from a purely normalization-oriented point of view, since they can be recovered through **Participation**, but they make graph traversal and query writing more convenient.

The `workedWith` relation between persons who collaborated on the same work is not materialised in the mapping. It is derivable at query time via shared **Participation** nodes:

```
SELECT DISTINCT ?p1 ?p2 WHERE {
  ?par1 ce:playedBy ?p1 ; ce:participatesIn ?t .
  ?par2 ce:playedBy ?p2 ; ce:participatesIn ?t .
  FILTER(?p1 != ?p2)
}
```

Materialising this relation via a self-join on `title_principal` would generate $O(n^2)$ triples and produce both directions of each pair, which is redundant given the `owl:SymmetricProperty` declaration on `ce:workedWith`.

Language and region information is derived directly from `title_aka` using the ISO codes stored in the `LANGUAGE_ID` and `REGION_ID` columns. Language and region are attached as ISO-coded string attributes to `CreativeWork` instances via `ce:language` and `ce:region`; the relational **Language** and **Region** lookup entities are not promoted to **RDF** classes, since the queries in Section 7 treat them as filter codes rather than entities to navigate.

The generated knowledge graph is serialised to Turtle format and loaded into an Apache Fuseki triplestore for persistent storage and **SPARQL** querying, as described in Section 6. Because the ontology targets the **OWL 2 QL** profile (Section 4.3), the relevant reasoning closures (subclass, sub-property, inverse, domain/range) admit query rewriting and can be applied at query time without materialising all inferred triples beforehand; alternatively, a reasoner can be applied offline to compute closures, or queries can rely on **SPARQL** property paths for traversal that would otherwise require inference.

6 Deployment

6.1 Architecture

The deployment consists of two independent Docker containers serving different purposes. The first container, defined in `database/docker-compose.yml`, runs MySQL 8 together with phpMyAdmin and is used exclusively during the **R2RML** mapping phase to expose the source relational database on port 3307. The second set of containers, defined in `deployment/docker-compose.yml`, runs two services: Apache Fuseki 5.5.0 as the persistent **SPARQL** triplestore on port 3030, and Brwsr as a Linked Data browser on port 5000. The two compose files are deliberately kept separate: MySQL is only needed when regenerating the knowledge graph from the relational source; Fuseki and Brwsr are used for all subsequent **SPARQL** querying, graph navigation, and demonstration activities.

6.2 Triplestore Setup

Apache Fuseki is an **RDF** triplestore and **SPARQL** 1.1 endpoint that is part of the Apache Jena framework. It supports persistent storage via TDB2, an on-disk quad store optimised for large **RDF** datasets. The Fuseki container is started with:

```
cd deployment && docker compose up -d
```

Once running, a persistent TDB2 dataset named `cineexplorer` is created through the Fuseki administration API:

```
curl -u admin:admin -X POST http://localhost:3030/$/datasets \
  -d "dbName=cineexplorer&dbType=tdb2"
```

6.3 Loading the Knowledge Graph

Both the ontology and the instance data are loaded into the default graph of the `cineexplorer` dataset. Loading the ontology alongside the instance data makes class and property declarations available in the same graph, so that **SPARQL** queries can reference `ce:` terms directly without requiring a `FROM` clause:

```
# Load ontology (314 triples)
curl -u admin:admin -X POST \
  http://localhost:3030/cineexplorer/data \
  --data-binary @ontology/cineexplorer_ontology.ttl \
  -H "Content-Type: text/turtle"
```

```
# Load knowledge graph (~1.6M triples)
curl -u admin:admin -X POST \
  http://localhost:3030/cineexplorer/data \
  --data-binary @output/cineexplorer_kg.ttl \
  -H "Content-Type: text/turtle"
```

After loading, the dataset contains 1,611,676 triples in total, generated from the global numVotes-ranked top-5,000 slice of the official **IMDb** non-commercial dataset (see Section 2). Table 6.3 shows the breakdown by class.

Class	Instances
ce:Participation	105,964
ce:Person	38,067
ce:Actor	25,693
ce:Writer	11,103
ce:Director	7,087
ce:Film	4,214
ce:Editor	1,904
ce:Composer	1,776
ce:Series	622
ce:Episode	139
ce:Genre	26

Table 6.3: **KG** instance counts by class. **Person** subclasses (**Actor**, **Writer**, **Director**, **Editor**, **Composer**) overlap, so their sum exceeds the total **Person** count.

6.4 SPARQL Endpoint

The **SPARQL** 1.1 query endpoint is available at:

```
http://localhost:3030/cineexplorer/query
```

The Fuseki web interface, accessible at `http://localhost:3030`, provides a browser-based query editor, dataset management, and upload utilities. All **SPARQL** queries described in Section 7 were executed against this endpoint.

6.5 Linked Data Browser

Brwsr is a lightweight Linked Data browser implemented in Python [4]. It acts as a frontend layer on top of the Fuseki **SPARQL** endpoint, implementing content negotiation: when an HTTP client requests a CineExplorer **IRI** with **Accept: text/html**, Brwsr returns a human-readable HTML page showing all triples for that resource with clickable hyperlinks; when the client requests **Accept: text/turtle** or **application/rdf+xml**, it returns the raw **RDF** serialisation. This makes every resource in the knowledge graph dereferenceable according to the Linked Data principles [11].

Brwsr is configured to serve resources under the `http://cineexplorer.local/` namespace by mapping requests to the Fuseki **SPARQL** endpoint at `http://localhost:3030/cineexplorer/query`. The browser is accessible at:

`http://localhost:5000`

Navigating to a person **IRI**, for example `http://localhost:5000/data/person/nm0000102`, returns a page listing all properties of that person — name, roles, titles worked on — with each linked resource rendered as a clickable hyperlink. This allows graph exploration without writing any **SPARQL**.

6.6 Named Graph Strategy

Both the ontology and the instance data are loaded into the default graph rather than into separate named graphs. This simplifies query writing: **SPARQL** queries do not need **FROM** or **FROM NAMED** clauses to access either the ontology terms or the instance data. An alternative design would place the ontology in a dedicated named graph (e.g., `<urn:cineexplorer:ontology>`) to keep the two layers distinct; however, for the scale of this project the default graph approach is sufficient.

7 SPARQL Queries

All queries were executed against the Apache Fuseki **SPARQL** 1.1 endpoint described in Section 6 (`http://localhost:3030/cineexplorer/query`). Query files and **CSV** result exports are available in the `sparql/` directory of the repository.

7.1 Query Overview

Table 7.4 maps each query to the **SPARQL** 1.1 feature it demonstrates.

Query	File	SPARQL feature(s)
Q1	<code>q01_films_genres.sparql</code>	SELECT, multi-join, ORDER BY
Q2	<code>q02_roles_count.sparql</code>	COUNT DISTINCT, GROUP BY, HAVING
Q3	<code>q03_worked_together.sparql</code>	Subquery, COUNT DISTINCT, ORDER BY, LIMIT
Q4	<code>q04_actors_multi_genre.sparql</code>	Subquery with HAVING, outer GROUP BY
Q5	<code>q05_no_director.sparql</code>	UNION, FILTER NOT EXISTS (negation)
Q6	<code>q06_bacon_number.sparql</code>	Property paths (<code>/</code> , <code>{1,2}</code>)
Q7	<code>q07_wikidata_federation.sparql</code>	SERVICE (federated query), BIND, REPLACE
Q8	<code>q08_series_seasons.sparql</code>	COUNT DISTINCT, OPTIONAL, GROUP BY
Q9	<code>q09_director_never_actor.sparql</code>	MINUS (set subtraction)
Q10	<code>q10_similar_titles.sparql</code>	Subquery, multi-join, COUNT DISTINCT

Table 7.4: **SPARQL** queries and the **SPARQL** 1.1 features they demonstrate.

7.2 Q1 — Films with Genres and Runtime

This query retrieves all films in the knowledge graph together with their genres and runtime, ordered from longest to shortest. It demonstrates multi-property joins across the `ce:Film` $\rightarrow$ `ce:Genre` chain and serves as a baseline sanity check for the deployment.

```
PREFIX ce: <http://cineexplorer.local/ontology#>
```

```
SELECT ?title ?genre ?runtime
WHERE {
  ?film a ce:Film ;
        ce:workTitle ?title ;
        ce:runtimeMinutes ?runtime ;
        ce:hasGenre ?g .
  ?g ce:genreName ?genre .
}
ORDER BY DESC(?runtime)
```

The query returns 11,252 rows (each film appears once per genre, since `ce:hasGenre` is multi-valued). The three longest films are Gangs of Wasseypur (321 min, Action/Comedy/Crime), Zack Snyder's Justice League (242 min), and Gone with the Wind (238 min).

7.3 Q2 — Persons per Participation Role

This query counts how many distinct persons hold each participation role and filters to roles with more than five persons. It demonstrates `COUNT DISTINCT`, `GROUP BY`, and `HAVING`.

```
PREFIX ce: <http://cineexplorer.local/ontology#>
```

```
SELECT ?role (COUNT(DISTINCT ?person) AS ?personCount)
WHERE {
  ?par a ce:Participation ;
        ce:participationRole ?role ;
        ce:playedBy ?person .
}
GROUP BY ?role
HAVING (COUNT(DISTINCT ?person) > 5)
ORDER BY DESC(?personCount)
```

Role	Persons
actor	22,705
writer	5,383
producer	4,093
director	1,825
editor	1,734
composer	1,308
production_designer	1,303
cinematographer	1,261
casting_director	1,092
self	333
archive_footage	47

Table 7.5: Q2 result: participation roles with more than 5 distinct persons.

The result (Table 7.5) shows eleven roles above the threshold. Note that **actor** subsumes both male and female performers in the migrated dataset: at **ETL** time the **IMDb** categories **actor** and **actress** are collapsed into a single **actor** value, so the 22,705 figure here covers what **IMDb** originally labels as separate categories.

7.4 Q3 — Persons Who Worked Together Most

This query identifies the top 10 pairs of persons who collaborated on the most shared titles. The inner subquery aggregates and ranks all pairs; the outer query resolves their names. `FILTER(STR(?p1) < STR(?p2))` ensures each pair is counted once rather than both (A, B) and (B, A).

```
PREFIX ce: <http://cineexplorer.local/ontology#>

SELECT ?name1 ?name2 ?sharedTitles
WHERE {
  {
    SELECT ?p1 ?p2 (COUNT(DISTINCT ?title) AS ?sharedTitles)
    WHERE {
      ?p1 ce:workedFor ?title .
      ?p2 ce:workedFor ?title .
      FILTER(STR(?p1) < STR(?p2))
    }
    GROUP BY ?p1 ?p2
    ORDER BY DESC(?sharedTitles)
    LIMIT 10
  }
  ?p1 ce:personName ?name1 .
  ?p2 ce:personName ?name2 .
}
ORDER BY DESC(?sharedTitles)
```

The top-10 pairs are dominated by long-running casting director and producer partnerships, who repeatedly co-credit on the same productions. The first three rows are Deborah Aquila with Tricia Wood (63 shared titles, both casting directors), Janet Hirshenson with Jane Jenkins (55 titles), and Tim Bevan with Eric Fellner (52 titles, the founders of Working Title Films). The migrated **KG** — where each person averages 2.78 credits across the slice — supports collaboration counts well above one, which the previous curated sample of 174 titles could not.

7.5 Q4 — Actors Spanning Three or More Genres

This query finds actors who appeared in titles covering at least three different genres. A subquery with `HAVING` first identifies qualifying actors; the outer query counts their genres. Using a subquery avoids computing genre counts for all 38,067 persons before filtering.

```
PREFIX ce: <http://cineexplorer.local/ontology#>

SELECT ?name (COUNT(DISTINCT ?genre) AS ?genreCount)
WHERE {
  {
    SELECT ?person
    WHERE {
      ?person a ce:Actor .
      ?person ce:actedIn ?title .
      ?title ce:hasGenre ?g .
      ?g ce:genreName ?genre .
    }
    GROUP BY ?person
    HAVING (COUNT(DISTINCT ?genre) >= 3)
  }
  ?person ce:personName ?name .
  ?person ce:actedIn ?title .
  ?title ce:hasGenre ?g .
  ?g ce:genreName ?genre .
}
```

```
GROUP BY ?person ?name
ORDER BY DESC(?genreCount)
```

The query returns 17,832 actor-genre rows: at the genre-spanning threshold of three, the migrated **KG** yields a wide set of qualifying actors (the row count is one row per actor-genre pair retained for the outer COUNT). The top of the result — actors spanning the highest number of distinct genres — is dominated by veteran performers with long **IMDb** careers.

7.6 Q5 — Titles with No Director Credited

This query identifies films and series that have no director credited in the knowledge graph, using `FILTER NOT EXISTS` as negation as failure.

```
PREFIX ce: <http://cineexplorer.local/ontology#>

SELECT ?title ?type
WHERE {
  { ?work a ce:Film . BIND("Film" AS ?type) }
  UNION
  { ?work a ce:Series . BIND("Series" AS ?type) }
  ?work ce:workTitle ?title .
  FILTER NOT EXISTS {
    ?person ce:directed ?work .
  }
}
ORDER BY ?type ?title
```

The query returns 626 titles (predominantly TV series) without a credited director. TV series frequently list directors only at the episode level rather than at the series level, which explains the dominance of `Series` rows in the output. Under the Open World Assumption, the absence of a `ce:directed` triple does not mean the work has no director — only that this information is not represented in the knowledge graph.

7.7 Q6 — Collaboration Distance via Property Paths

This query explores the collaboration graph using **SPARQL** 1.1 property paths. Russell Crowe (`nm0000128`) is used as the source for two reasons: he is well represented in the migrated **KG** (558 direct collaborators across his credited filmography in the slice), and he is one of the historical anchors of the project's running example through *Les Misérables* (2012).

Direct collaborators (1-hop):

```
PREFIX ce: <http://cineexplorer.local/ontology#>

SELECT DISTINCT ?collaboratorName
WHERE {
  BIND(<http://cineexplorer.local/data/person/nm0000128> AS ?source)
  ?source ce:workedFor ?title .
  ?collaborator ce:workedFor ?title .
  FILTER(?collaborator != ?source)
  ?collaborator ce:personName ?collaboratorName .
}
ORDER BY ?collaboratorName
```

Persons reachable within 2 hops:

```
PREFIX ce: <http://cineexplorer.local/ontology#>
```

```

SELECT DISTINCT ?reachableName
WHERE {
  BIND(<http://cineexplorer.local/data/person/nm0000128> AS ?source)
  ?source (ce:workedFor/ce:employed){1,2} ?reachable .
  FILTER(?reachable != ?source)
  ?reachable ce:personName ?reachableName .
}
ORDER BY ?reachableName

```

The path `(ce:workedFor/ce:employed){1,2}` traverses one or two hops, where each hop corresponds to a shared title. The 1-hop query returns 558 direct collaborators of Russell Crowe; the 2-hop query returns 21,214 distinct persons reachable within two collaborations. The $38\times$ growth from 1-hop to 2-hop is the headline structural property of the migrated dataset — the previous curated sample of 174 titles had no path-multiplying intermediaries because no person was credited on more than one title.

7.8 Q7 — Wikidata Federation

This query enriches CineExplorer persons with birth dates from Wikidata via a federated `SERVICE` call. Wikidata property P345 stores **IMDb** person identifiers (`nconst`), which are embedded in our person **IRIs**. The `BIND(REPLACE(...))` pattern extracts the `nconst` at query time without materialising any `sameAsExternal` triples, demonstrating how Linked Data **IRIs** designed as Cool **URIs** [11] can be re-used for federation across endpoints.

```

PREFIX ce: <http://cineexplorer.local/ontology#>
PREFIX wdt: <http://www.wikidata.org/prop/direct/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?localName ?nconst ?wikidataItem ?wikidataLabel ?birthDate
WHERE {
  VALUES ?person { <.../nm0000128> <.../nm0004266> ... } # 10 persons
  ?person ce:personName ?localName .
  BIND(REPLACE(STR(?person), "^.*/", "") AS ?nconst)
  SERVICE <https://query.wikidata.org/sparql> {
    ?wikidataItem wdt:P345 ?nconst .
    OPTIONAL { ?wikidataItem wdt:P569 ?birthDate . }
    OPTIONAL {
      ?wikidataItem rdfs:label ?wikidataLabel .
      FILTER(LANG(?wikidataLabel) = "en")
    }
  }
}
ORDER BY ?localName

```

Local name	nconst	Wikidata	Born
Alain Boublil	nm0098842	Q284540	1941-03-05
Amanda Seyfried	nm1086543	Q189226	1985-12-03
Anne Hathaway	nm0004266	Q36301	1982-11-12
Claude-Michel Schönberg	nm0774744	Q712004	1944-07-06
Herbert Kretzmer	nm0471014	Q5734832	1925-10-05
Hugh Jackman	nm0413168	Q129591	1968-10-12
Russell Crowe	nm0000128	Q129817	1964-04-07
Tom Hooper	nm0393799	Q295912	1972-10-05
William Nicholson	nm0629933	Q706935	1948-01-12

Table 7.6: Q7 result: CineExplorer persons enriched with Wikidata birth dates (9 of 10 returned).

Nine of the ten persons in the input `VALUES` list match (Table 7.6). The tenth, Victor Hugo

(nm0401076, Wikidata Q535), is correctly resolved on the Wikidata side — but does not appear in our local **KG** slice, so the join on `ce:personName` drops the row. This is the expected behaviour of a federated query: the local **KG** anchors the join, and the `SERVICE` call enriches only those local resources that already exist. The match rate among persons that are in the slice is therefore 100%, reflecting Wikidata’s comprehensive coverage of major film productions.

7.9 Q8 — Seasons and Episodes per Series

This query counts distinct seasons and episodes per series. The `OPTIONAL` clause handles episodes that lack a `ce:seasonNumber` value, producing `seasonCount = 0` for those rows rather than excluding them.

```
PREFIX ce: <http://cineexplorer.local/ontology#>

SELECT ?seriesTitle
      (COUNT(DISTINCT ?season) AS ?seasonCount)
      (COUNT(DISTINCT ?ep) AS ?episodeCount)
WHERE {
  ?series a ce:Series ;
          ce:workTitle ?seriesTitle ;
          ce:hasEpisode ?ep .
  OPTIONAL { ?ep ce:seasonNumber ?season . }
}
GROUP BY ?series ?seriesTitle
ORDER BY DESC(?episodeCount)
```

The query returns 16 series in the slice that have at least one linked episode. The migrated **KG** contains 622 series and 139 episodes overall, but most series have only a small number of their episodes carried into the slice — the slice is selected from `title.basics` by `numVotes` per title type, so only the most-voted-on individual episodes survive. The result is therefore dominated by a few cult series for which several episodes cleared the popularity threshold (e.g., Game of Thrones, Breaking Bad). The `OPTIONAL` clause cleanly returns `seasonCount = 0` for episodes without a `ce:seasonNumber` value, demonstrating the gradual handling of missing properties under the Open World Assumption (**OWA**).

7.10 Q9 — Directors Who Never Acted

This query uses `MINUS` to subtract the set of directors who have an acting credit from the full set of directors.

```
PREFIX ce: <http://cineexplorer.local/ontology#>

SELECT ?name
WHERE {
  ?person a ce:Director ;
          ce:personName ?name .
  MINUS {
    ?person ce:actedIn ?anyTitle .
  }
}
ORDER BY ?name
```

`MINUS` operates on solution sets (whole result rows), while `FILTER NOT EXISTS` operates on variable bindings. For this pattern the two forms produce identical results, but `MINUS` expresses the intent more directly: remove from the director set those persons for whom an `actedIn` triple exists. The query returns 3,580 persons — directors with no acting credit in the knowledge graph. The complementary set (directors who also acted) is $7,087 - 3,580 = 3,507$, which agrees with the two-axis modelling of the `primaryProfession` attribute and the per-credit `Participation` role: a director may be typed as

`ce:Director` either by Participation evidence (a `director` credit on some title) or by their **IMDb**-level career profession, and the same person can also accrue acting credits, which the query correctly excludes.

7.11 Q10 — Titles Sharing Genre and Language

Given Les Misérables as target, this query finds titles that share at least one genre and at least one language with it, ranked by the number of shared genres. An inner subquery identifies language-compatible candidates; the outer query then restricts to those sharing a genre and counts shared genres.

```
PREFIX ce: <http://cineexplorer.local/ontology#>

SELECT ?candidateTitle (COUNT(DISTINCT ?sharedGenre) AS ?commonGenres)
WHERE {
  BIND(<http://cineexplorer.local/data/title/tt1707386> AS ?target)
  ?target ce:hasGenre ?sharedGenre .
  ?candidate ce:hasGenre ?sharedGenre .
  FILTER(?candidate != ?target)
  ?candidate ce:workTitle ?candidateTitle .
  {
    SELECT DISTINCT ?candidate WHERE {
      BIND(<http://cineexplorer.local/data/title/tt1707386> AS ?target)
      ?target ce:language ?lang .
      ?candidate ce:language ?lang .
    }
  }
}
GROUP BY ?candidate ?candidateTitle
ORDER BY DESC(?commonGenres)
```

With Les Misérables as target (3 genres, 7 languages), the query returns 2,751 candidates ranked by number of shared genres. The dense scale of the migrated **KG** — where many films share Drama as a genre and English or French as a language — means a far larger candidate set than the curated sample produced. The subquery structure separates the language filter (inner) from the genre count (outer), demonstrating how subqueries can decompose multi-condition similarity patterns into reusable layers.

8 Non-trivial Demonstrator

This section presents two non-trivial demonstrations built on top of the CineExplorer knowledge graph: a collaboration-graph analysis based on the Bacon number concept, and a **SHACL** structural validation that contrasts closed-world constraint checking with the open-world semantics of the **OWL** ontology. Together they exercise both directions of “going beyond simple SELECT queries”: graph-shaped analytics that would be awkward in **SQL**, and meta-level reasoning about the graph itself.

8.1 Bacon Number — Collaboration Graph Analysis

8.1.1 Concept

The Six Degrees of Kevin Bacon game assigns a “Bacon number” to each actor: 0 for Kevin Bacon himself, 1 for anyone who appeared in a film with him, 2 for anyone who appeared with a Bacon-number-1 actor, and so on. In graph terms, this is the length of the shortest path in the collaboration graph, where nodes are persons and edges connect persons who worked on the same title.

In the CineExplorer **KG**, collaboration paths are expressed with **SPARQL** 1.1 property paths. Each hop traverses one shared title: `ce:workedFor` goes from a person to a creative work, and `ce:employed` goes back from the work to another person. The path `(ce:workedFor/ce:employed){1,n}` finds all persons reachable within 1 to n hops. Iterating on n yields the Bacon-number ball expanding around the source.

8.1.2 Source Person

We use Kevin Bacon himself (nm0000102) as the anchor for the canonical demonstration — both because it preserves the original game and because the migrated **KG** contains his full filmography slice (his profession matches the **numVotes** top-ranking criterion of Section 2). For comparison we also report the same expansion starting from Russell Crowe (nm0000128), the project’s running example through *Les Misérables* (2012).

8.1.3 Bacon Number Results

Table 8.7 reports the size of the reachable set $|R_n|$ at hop counts $n \in \{1, 2, 3\}$, expressed both in absolute terms and as a percentage of the 38,067 persons in the **KG**.

Source	1-hop	2-hop	3-hop	Coverage @3
Kevin Bacon (nm0000102)	403	18,155	35,033	92.0%
Russell Crowe (nm0000128)	558	21,214	—	55.7% @2

Table 8.7: Bacon-number expansion from two source persons. Each column counts distinct persons reachable within n hops via $(\text{ce:workedFor}/\text{ce:employed})\{1, n\}$, excluding the source. Coverage @3 is computed against the 38,067 **ce:Person** instances in the **KG**.

The Kevin Bacon ball expands roughly exponentially: $\sim 45\times$ from 1-hop to 2-hop, then a further $\sim 2\times$ to 3-hop where it covers 92% of the entire person catalogue. By three hops, 35,033 of the 38,067 persons in the **KG** are reachable from Kevin Bacon; only 3,034 remain unreachable, almost all of them on isolated short / video / videoGame titles whose participations did not overlap with the main film cluster (see Section 8.2.4).

This is the structural property that motivated migrating away from the previous curated sample of 174 titles. In that earlier dataset, no person appeared on more than one title and the collaboration graph was a disjoint union of one cluster per title; the 1-hop and 2-hop neighbourhoods coincided trivially because no intermediary person could span two productions. The migrated **KG**, with each person averaging 2.78 credits, recovers the expanding-neighbourhood behaviour that gives the Bacon-number game its meaning.

8.1.4 Path Reconstruction

To make the path explicit, we can recover the bridge title between any two persons:

```
PREFIX ce: <http://cineexplorer.local/ontology#>

SELECT ?bridgeTitle WHERE {
  BIND(<http://cineexplorer.local/data/person/nm0000102> AS ?p1)
  BIND(<http://cineexplorer.local/data/person/nm0000128> AS ?p2)
  ?p1 ce:workedFor ?w .
  ?p2 ce:workedFor ?w .
  ?w ce:workTitle ?bridgeTitle .
}
```

Variants of the same query template, parameterised over **?p1** and **?p2**, also recover full n -hop paths by chaining **ce:workedFor/ce:employed** multiple times in a single Basic Graph Pattern (**BGP**). Since a Bacon-number computation is essentially a shortest-path query over the collaboration graph, this demonstrates that the **KG** supports graph-shaped analytics natively, without requiring an external graph engine or a separately materialised collaboration relation.

8.2 SHACL Structural Validation

8.2.1 SHACL vs OWL Cardinality

SHACL (Shapes Constraint Language) is a W3C standard for expressing structural constraints on **RDF** graphs and validating instances against them. Unlike **OWL** cardinality restrictions, which operate

under the **OWA** (an absent triple does not mean a constraint is violated), **SHACL** operates under the Closed World Assumption (**CWA**): a missing required triple is reported as a violation.

The CineExplorer ontology already encodes structural intent through **OWL** axioms (minimum cardinalities, disjointness). **SHACL** complements this by enabling runtime validation of the generated knowledge graph against explicitly stated structural shapes, which lets us detect data-quality issues that **OWL** alone would silently allow.

8.2.2 Shapes Defined

Five `NodeShapes` were defined in `sparql/cineexplorer_shapes.ttl`:

1. **CreativeWorkShape** — every `ce:CreativeWork` must have exactly one `ce:workTitle` (`xsd:string`), exactly one `ce:averageRating` (`xsd:decimal` in `[1.0,10.0]`), and exactly one `ce:numVotes` (`xsd:nonNegativeInteger`).
2. **PersonShape** — every `ce:Person` must have exactly one `ce:personName`; `ce:hasProfession` is optional, with at most three values per person (matching **IMDb**'s `primaryProfession` field).
3. **EpisodeShape** — every `ce:Episode` must link to exactly one `ce:Series` via `ce:partOfSeries`.
4. **ParticipationShape** — every `ce:Participation` must link to exactly one `ce:Person` via `ce:playedBy`, exactly one `ce:CreativeWork` via `ce:participatesIn`, and have at least one `ce:participationRole` string.
5. **GenreShape** — every `ce:Genre` must have exactly one `ce:genreName` (`xsd:string`).

The mandatory `ce:averageRating` and `ce:numVotes` constraints follow directly from the slice criterion of Section 2: every title carried into the **KG** was selected by `numVotes` ranking, so each title is expected to carry both values. This is a textbook case where the **SHACL** shape captures a real integrity invariant of the **ETL** pipeline.

8.2.3 Validation Results

Validation was performed with `pyshacl` 0.31.0 in two configurations.

Run 1 — without ontology inference. The shapes file is loaded together with the data graph; no ontology axioms are added.

```
pyshacl -s sparql/cineexplorer_shapes.ttl \  
        -d output/cineexplorer_kg.ttl \  
        -f human  
-- Conforms: False  
-- Results (105,964 violations)
```

Run 2 — with **RDFS inference on (ontology + data).** The ontology is provided as a separate graph, and `pyshacl` materialises the **RDFS** entailment closure before validation.

```
pyshacl -s sparql/cineexplorer_shapes.ttl \  
        -e ontology/cineexplorer_ontology.ttl \  
        -i rdfs \  
        -d output/cineexplorer_kg.ttl \  
        -f human  
-- Conforms: False  
-- Results (75 violations across 25 focus nodes)
```

The two runs differ by more than three orders of magnitude. The cause is the same constraint in both runs — `sh:class ce:CreativeWork` on `ParticipationShape`'s `ce:participatesIn` path — but the population of focus nodes that satisfy it changes radically once subclass reasoning is applied. In Run 1, every `ce:Participation` (105,964 instances) reports a violation because the value of its `ce:participatesIn` edge is typed only as `ce:Film`, `ce:Series`, or `ce:Episode` — never as `ce:CreativeWork` directly — and `pyshacl` without inference cannot see that `ce:Film rdfs:subClassOf ce:CreativeWork`. In Run 2, **RDFS** entailment populates the missing `rdf:type` triples and that constraint is satisfied for every `Participation`. The $105,964 \rightarrow 75$ reduction is a clean illustration of the **OWA** / **CWA** trade-off in real numbers.

8.2.4 Discussion

Run 2 still reports 75 violations across 25 distinct focus nodes (Table 8.8). These are not noise: they identify a real, otherwise-invisible mapping gap that the migration introduced.

Constraint message (truncated)	Violations
Every CreativeWork must have exactly one <code>ce:workTitle</code> .	25
Every CreativeWork must have exactly one <code>ce:averageRating</code> in [1.0, 10.0].	25
Every CreativeWork must have exactly one <code>ce:numVotes</code> ≥ 0 .	25

Table 8.8: Run 2 SHACL violations. All 75 violations come from the same 25 focus nodes, each missing the three mandatory `CreativeWork` attributes.

The 25 focus nodes are titles whose `rdf:type` is inferred to `ce:CreativeWork` (because they appear as the object of `ce:participatesIn` or `ce:knownFor`, both of which have `rdfs:range ce:CreativeWork`) but for which no triple-map ever emitted core attributes. Tracing back to the relational source, all 25 belong to `IMDb` `titleType` values that the `R2RML` mapping does not cover: `video` (9), `short` (5), `tvMovie` (5), `tvSpecial` (3), and `videoGame` (3). The mapping covers only `movie` $\rightarrow$ `ce:Film`, `tvSeries/tvMiniSeries/tvPilot` $\rightarrow$ `ce:Series`, and `tvEpisode` $\rightarrow$ `ce:Episode`; the remaining types pass through to the database but produce no class-level `RDF`. They are still referenced from the Person side — a `Participation` on a `tvSpecial`, or a `ce:knownFor` link to a `video`, both produce object `IRIs` that point to those titles — so under `RDFS` inference the dangling references appear as bare `ce:CreativeWork` instances with no attributes.

This finding illustrates the value of `SHACL` for `KG` quality assurance. `OWL` alone, under the `OWA`, would treat the missing `ce:workTitle` / `ce:averageRating` / `ce:numVotes` triples as “not yet known” and raise no contradiction — the dangling references would remain invisible. `SHACL` with `RDFS` inference makes the integrity gap explicit and points directly at the mapping coverage to fix. The two runs together correspond to two different Knowledge Representation and Reasoning (`KRR`) perspectives on the same data: `OWL`’s open-world view, in which absence is non-information, and `SHACL`’s closed-world view, in which absence is the report.

8.3 Summary

The two demonstrators exercise complementary capabilities of the deployment. The Bacon-number analysis shows that the migrated `KG`’s collaboration graph is rich enough to support meaningful path queries, with a Kevin-Bacon ball that covers 92% of all persons within three hops. The `SHACL` validation shows that the ontology’s structural intent can be turned into actionable integrity checks, with the two-run comparison surfacing both the `OWA` / `CWA` distinction at the conceptual level and a concrete 25-title mapping gap at the data level. Both demonstrators are reproducible from the artefacts in the repository — the `SPARQL` queries in `sparql/q06.bacon_number.sparql` and `sparql/q06b.bacon.2hops.sparql`, and the `SHACL` shapes in `sparql/cineexplorer_shapes.ttl`.

9 Conclusion

This report described the full pipeline from a relational `IMDb` source to a deployed and queryable knowledge graph. Milestone 1 produced a 14-relation schema reduced from a conceptual `ERD` and populated from a numVotes-ranked top- N slice of the official `IMDb` non-commercial dataset. Milestone 2 produced an `OWL 2 DL` ontology centred on `CreativeWork` as the abstraction of all titles, with a reified `Participation` class capturing per-credit context. Milestone 3 connected the two layers via an `R2RML` mapping that produced a 1,611,676-triple knowledge graph. Milestone 4 deployed the result in Apache Fuseki, evaluated a battery of `SPARQL` queries covering subqueries, federation, property paths, aggregates, and negation, and presented two non-trivial demonstrators built on top.

Two findings stand out. First, the migration to the official `IMDb` dataset transformed the collaboration graph from a disjoint union of one cluster per title into a densely connected component: a Bacon-number ball expanding from Kevin Bacon covers 92% of all 38,067 persons within three hops, recovering the structural property that gives the demonstrator its meaning. Second, the `SHACL` validation surfaced a real mapping gap that the ontology alone could not catch: 25 titles whose `titleType`

value (`short`, `video`, `tvMovie`, `tvSpecial`, `videoGame`) is not covered by the **R2RML** mapping appear as “phantom” **CreativeWork** instances under **RDFS** inference. The two **SHACL** runs — without and with inference — give a clean numerical illustration of the **OWA** / **CWA** contrast: 105,964 violations under pure **CWA** vs. 75 violations once subclass reasoning is applied, with the latter pointing at a specific, actionable data-quality issue.

Several follow-up directions remain open. The mapping could be extended to type the missing `titleType` values, eliminating the phantom **CreativeWorks**. The Bacon-number demonstrator could be packaged as a public-facing widget over the Brwsr deployment. Federation could go beyond Wikidata enrichment of birth dates towards constructing combined queries across multiple endpoints. The **KG** itself, once stable, becomes a natural substrate for downstream applications such as recommendation, visualisation, or natural-language-to-**SPARQL** interfaces.

10 Member Contributions

Although each member focused on specific tasks (Table 10.9), we worked collaboratively during the review and editing phases.

Table 10.9: Member contributions.

Name	Contribution
Dawid	
Hoang Linh	
Duy Vu	

References

- [1] R. Cyganiak, A. Das, and J. Sequeda. R2RML: RDB to RDF Mapping Language. <https://www.w3.org/TR/r2rml/>, 2012. W3C Recommendation, 27 September 2012.
- [2] B. Glimm, I. Horrocks, B. Motik, G. Stoilos, and Z. Wang. HermiT: An OWL 2 Reasoner. In *Journal of Automated Reasoning*, volume 53, pages 245–269. Springer, 2014.
- [3] P. Hitzler, M. Krötzsch, B. Parsia, P. F. Patel-Schneider, and S. Rudolph. OWL 2 Web Ontology Language Primer (Second Edition). W3c recommendation, World Wide Web Consortium (W3C), 2012.
- [4] R. Hoekstra et al. Brwsr: Lightweight Linked Data Browser. <https://github.com/Data2Semantics/brwsr>, 2014. GitHub repository, clariah/brwsr Docker image.
- [5] IMDb. Can i use imdb data in my software? <https://help.imdb.com/article/imdb/general-information/can-i-use-imdb-data-in-my-software/G5JTRESSHJBBHTGX>. Accessed: 2026-02-22.
- [6] IMDb. Imdb non-commercial datasets. <https://developer.imdb.com/non-commercial-datasets/>, 2025. Accessed: 2026-02-26.
- [7] S. Lohmann, S. Negru, F. Haag, and T. Ertl. Webvowl: Web-based visualization of ontologies. In *Proceedings of the 19th International Conference on Knowledge Engineering and Knowledge Management*, 2014.
- [8] B. Motik, B. C. Grau, I. Horrocks, Z. Wu, A. Fokoue, and C. Lutz. OWL 2 Web Ontology Language: Profiles (Second Edition). <https://www.w3.org/TR/owl2-profiles/>, 2012. W3C Recommendation, 11 December 2012.
- [9] northCoder. Using imdb as a test data set. <https://northcoder.com/post/using-imdb-as-test-data-set>, 2019. Accessed: 2026-02-22.
- [10] northCoder repo. Sample imdb data for relational databases. for testing/demo purposes only. <https://github.com/northcoder-repo/relational-sample-imdb-data>, 2019. Accessed: 2026-02-22.

- [11] L. Sauermann and R. Cyganiak. Cool URIs for the Semantic Web. W3c interest group note, World Wide Web Consortium (W3C), 2008.
- [12] Wikidata. Wikidata. <https://www.wikidata.org>, 2023. Accessed: 2026-03-05.

A Appendix: The use of AI

We used AI tools in a partially supportive way. ChatGPT was employed to explain references, summarize abstracts, and suggest content reorganization for better flow. DeepL and ChatGPT were used for grammar and language correction

B Additional Information on the Relational Model

This appendix supplements Section 3.2 with the modelling assumptions, attribute domains, and non-visible integrity constraints of the migrated CineExplorer relational schema. The 16 relations are listed in Section 3.2; the **ERD** they reduce from is in Section 3, Figure 3.1.

B.1 Modelling Assumptions and Cardinalities

Table B.10 summarises the main modelling assumptions and min-max cardinalities used in the **ERD**.

Table B.10: Key **ERD** assumptions and cardinalities for the migrated schema.

Element	Assumption / cardinality (min, max)
Title – Title.Type	Each Title has exactly one Title.Type: Title (1,1) to Title.Type (0,N). The <code>title_type_id</code> FK on <code>title</code> both classifies and serves as the IS-A discriminator.
Title – Genre (multi-valued attribute)	A Title may have zero or more genres: Title (0,N) to Genre name. Reduced to attribute relation <code>title_genre</code> (TCONST, <code>GENRE_NAME</code>); <code>genre_name</code> is part of the PK encoding the multi-valued attribute. The <code>Adult</code> value is excluded at ETL time; the boolean <code>is_adult</code> on <code>Title</code> carries that information.
Principal (Title, Person, Role)	The n-ary association is reified into the weak entity set Principal (named after IMDb ’s source vocabulary; the ontology layer calls the same concept <code>ce:Participation</code>). Identified by (<code>tconst</code> , <code>ordering</code>). Cardinalities: Title (0,N) to Principal (1,1); Person (0,N) to Principal (1,1); Role (0,N) to Principal (1,1). Each Principal carries <code>job</code> and a multi-valued <code>character_name</code> .
Title_AKA as weak entity set	Identified by Title plus local <code>ordering</code> . Identifying relationship: Title (0,N) to Title_AKA (1,1).
Title_AKA – Language / Region	Optional links: Title_AKA (0,1) to Language (0,N); Title_AKA (0,1) to Region (0,N).
Title_AKA – AKA_Type	M:N. One AKA may carry several types (e.g., <code>imdbDisplay</code> and <code>dvd</code>); IMDb stores them as byte-0x02-separated values. Cardinality: Title_AKA (0,N) to AKA_Type (0,N), reduced via the <code>title_aka_type</code> junction. <code>AKA_Type</code> is the only seeded lookup (its 8-value vocabulary is documented and stable, so each row carries a description as well as a name).
Series / Episode / Film (IS-A)	IS-A of <code>Title</code> , partial-disjoint, discriminator-driven. <code>Series</code> = {tvSeries, tvMiniSeries, tvPilot}; <code>Episode</code> = tvEpisode; <code>Film</code> = movie. Other <code>title_type_id</code> values remain plain <code>Title</code> rows. Only <code>Episode</code> stores extra attributes (in <code>title_episode</code>).
Series – Episode (hasEpisode)	Conceptually each Episode belongs to exactly one Series: Episode (1,1) to Series (1,N). The slice criterion may produce series with zero episodes in the slice, so the implemented cardinality is recorded as Episode (1,1) to Series (0,N). Season and episode numbers are stored in <code>title_episode</code> .
Person – Profession (hasProfession)	Career-level multi-valued attribute, capped at 3 per Person by IMDb : Person (0,3) to Profession (0,N). M:N junction <code>person_profession</code> .
Person – Title (knownFor)	Capped at 4 known-for titles per Person by IMDb : Person (0,4) to Title (0,N). M:N junction <code>person_known_for</code> , FK-closed at ETL time so all retained references resolve to the slice.

B.2 Notes on the Reduction

The four-group reduction listed in Section 3.2 resolves as follows. The six lookup relations (`Title.Type`, `Role`, `Profession`, `Language`, `Region`, `AKA_Type`) carry the bounded label sets used in the source **TSVs**; each is referenced by FKs from at least one weak entity set and may carry display attributes. Five are populated dynamically from the staging tables; `AKA_Type` is the exception, seeded with the 8 **IMDb**-documented codes plus a human-readable description column (the codes such as `imdbDisplay`, `dvd`, `working` are not self-describing). For the IS-A specialisation, only `Episode` carries extra attributes (`title_episode`); `Film` and `Series` are encoded entirely by the discriminator `title.title_type_id` and reduce to predicate views over `Title`. Both weak entities (`Title_AKA`, `Principal`) take their composite PK (`tconst`, `ordering`) directly from the source **TSVs**. Among the five multi-valued/junction reductions, `Title_Genre` and `Principal_Character` encode multi-valued attributes whose value is part of the PK, while `Person_Profession`, `Person_Known_For`, and `Title_AKA_Type` are true M:N junctions whose ends are both entity sets; `Title_AKA_Type` is itself the reduction of **IMDb**’s byte-0x02-separated multi-valued `title.akas.types` attribute — one row per (AKA, type) pair, with FKs into both `Title_AKA` (composite) and `AKA_Type`.

B.3 Attribute Domains

The attribute domains are inferred from the **IMDb** non-commercial dataset [6]. Each **tconst/nconst** string-PK is enforced by the natural-key pattern **tt\d+** / **nm\d+**. Numeric attributes use **SQL INT** where **IMDb** provides a year or a count, and **DECIMAL(3,1)** for **average_rating**. Non-numeric attributes use **VARCHAR** with capacity sufficient for the longest value observed in the slice (500 characters for primary/original titles, 100 for label-set names, 64 for shorter codes such as **role_name**).

- **Title**
 - $\text{dom}(\text{tconst})$ = strings matching pattern **tt\d+**
 - $\text{dom}(\text{primary_title}), \text{dom}(\text{original_title})$ = non-empty string
 - $\text{dom}(\text{is_adult}) = \{0, 1\}$
 - $\text{dom}(\text{start_year}), \text{dom}(\text{end_year}), \text{dom}(\text{runtime_minutes}) = \mathbb{N} \cup \{\text{null}\}$
 - $\text{dom}(\text{average_rating}) = [1.0, 10.0] \cup \{\text{null}\}$ (**IMDb**'s range)
 - $\text{dom}(\text{num_votes}) = \mathbb{N} \cup \{\text{null}\}$
 - $\text{dom}(\text{title_type_id})$ = bounded string set {movie, tvSeries, ...} (FK)
- **Person**
 - $\text{dom}(\text{nconst})$ = strings matching pattern **nm\d+**
 - $\text{dom}(\text{primary_name})$ = non-empty string
 - $\text{dom}(\text{birth_year}), \text{dom}(\text{death_year}) = \mathbb{N} \cup \{\text{null}\}$
- **Lookup entity sets** (**Title_Type**, **Role**, **Profession**, **Language**, **Region**)
 - Each has a string or INT PK plus a non-empty ***_name** attribute. **Language_id** and **Region_id** use ISO codes (alphanumeric strings); **title_type_id** uses **IMDb**'s enumeration (e.g., **movie**); **role_id** and **profession_id** are surrogate INT **AUTO_INCREMENT**.
- **Title_AKA** (weak entity set)
 - $\text{dom}(\text{ordering}) = \mathbb{N}^+$
 - $\text{dom}(\text{title})$ = non-empty string (the alternative title)
 - $\text{dom}(\text{is_original_title}) = \{0, 1\}$
 - $\text{dom}(\text{attributes}) = \text{string} \cup \{\text{null}\}$ (free-text qualifiers)
 - $\text{dom}(\text{region_id}), \text{dom}(\text{language_id}) = \text{ISO code} \cup \{\text{null}\}$ (FK)
- **Title_AKA_Type** (M:N junction)
 - $\text{dom}(\text{aka_type_id}) \in \{\text{imdbDisplay}, \text{original}, \text{alternative}, \text{tv}, \text{dvd}, \text{video}, \text{festival}, \text{working}\}$ (FK to seeded **AKA_Type** lookup).
- **Principal** (weak entity set)
 - $\text{dom}(\text{ordering}) = \mathbb{N}^+$
 - $\text{dom}(\text{role_id}) = \mathbb{N}^+$ (FK)
 - $\text{dom}(\text{job}) = \text{string} \cup \{\text{null}\}$
- **Title_Episode** (IS-A subtype)
 - $\text{dom}(\text{parent_tconst})$ = matches pattern **tt\d+** (FK to **Title**)
 - $\text{dom}(\text{season_number}), \text{dom}(\text{episode_number}) = \mathbb{N}^+ \cup \{\text{null}\}$
- **Title_Genre** (multi-valued attribute)
 - $\text{dom}(\text{genre_name})$ = bounded string set, 26 distinct values in the slice (Adult excluded).
- **Principal_Character** (multi-valued attribute)
 - $\text{dom}(\text{character_name})$ = non-empty string (after JSON-array unwrap from **title.principals.characters**).

B.4 Description of Non-visible Integrity Constraints

- If both `start_year` and `end_year` are present for a `Title`, then `start_year` $\leq$ `end_year`.
- If both `birth_year` and `death_year` are present for a `Person`, then `birth_year` $\leq$ `death_year`.
- If `average_rating` is present, its domain is $[1.0, 10.0]$ (IMDb's published range).
- For every retained `tconst` in `principal.tconst`, a corresponding row exists in `title`; for `principal.nconst`, a corresponding row exists in `person`. These are enforced as deferred FKs in `03_indexes.sql` after the staging-table population step.
- `person_known_for.tconst` references must resolve to titles within the slice. The closure is enforced at ETL time by INNER JOIN, before the FK is added.

C Appendix: The Earlier Curated Sample

This appendix documents the third-party curated CSV sample of IMDb used during Milestones M1–M3, and the structural reason it was retired in favour of the official non-commercial release at Milestone M4. It is included for completeness and to make the migration decision auditable; the body of the report (Section 2 onward) describes only the official source actually used in the deployed knowledge graph.

Source

The sample was distributed by northCoder [9, 10] as a set of CSV files derived from the official IMDb non-commercial release, repackaged as a teaching dataset for relational-database exercises. It mirrored a subset of the IMDb TSV layout, with one row per title, one row per person, and a junction file for per-credit information.

Scale

The curated sample contained:

- 174 titles
- 1,441 persons
- 586 principal credits
- 74 episodes
- 28 distinct genres

The relational design developed against this schema (entity sets, weak-entity decomposition, junction tables) carried over to the official-IMDb migration with only minor renaming, so the M1–M3 modelling work was not wasted.

Why It Was Retired

The sample was excellent for early ontology and mapping work: small enough to inspect end-to-end, large enough to exercise every triple map. It was retired at Milestone M4 because of a single structural property of the data, not the schema:

No person in the curated sample appeared in more than one title.

For the actor collaboration graph that underpins the Bacon-number demonstrator (Section 8), this property is fatal. With every person credited on at most one title, the collaboration graph degenerates into a disjoint union of single-title cliques: 1-hop and 2-hop neighbourhoods coincide, and there is no expansion to a Bacon centre or anywhere else. The headline use case the project committed to deliver could not be exercised on this data, however well-modelled.

Decision

We considered three responses: (i) hand-augment the sample with bridging credits, (ii) graft a second sample on top, or (iii) start over from the official **IMDb** non-commercial release. Option (iii) was chosen as the only one that preserved provenance and reproducibility: the official release is canonical, daily-refreshed, and licensed for non-commercial use [6, 5]. The **R2RML** mapping required only modest refactoring, and the relational schema needed minor renaming plus one new column to track **numVotes** (the slicing key introduced in Section 2.2).

A summary of the comparison that drove the decision is given in Table C.11.

Dimension	Curated CSV sample	Official IMDb non-commercial
Provenance	Third-party CSV repackage	Direct from IMDb
Refresh	Static snapshot	Daily
Coverage	174 titles, 1,441 persons	~12M titles, ~14M persons
Bridge persons	0	13,521 (in the 5,000-title slice)
Bacon demonstrator	Not realisable	Realisable (single connected component)
Reproducibility	Static	Deterministic via top- N numVotes

Table C.11: Comparison of the earlier curated sample and the official **IMDb** non-commercial dataset on the dimensions that drove the Milestone M4 migration decision.